

INDEX

SL. NO	TOPICS	SIGN
1.	Write a C program to generate n^{th} fibonacci number	
2.	Write a C program to reverse the given decimal number	
3.	Write a C program to convert binary number to decimal number	
4.	Write a C program to calculate $\sin(x)$ value without using inbuilt function	
5.	Write a C program to calculate $\cos(x)$ value without using inbuilt functions	
6.	Write a C program to calculate e^x value without using inbuilt functions	
7.	Write a C program to calculate GCD of two numbers	
8.	Write a C program to generate prime numbers	
9.	Write a C program to find square root of number without inbuilt function.	
10.	Write a C program to convert character to ascii code	
11.	Write a C program to partition of an array	
12.	Write a C program to remove duplicate elements from an array	
13.	Write a C program to count duplicate elements from an array	
14.	Write a C program to calculate prime factor of a number	

INDEX

[illegible]

Lab Program - 01.

① Write a C program to generate n^{th} fibonacci number.

Algorithm:

Step 1: Start

Step 2: Input n

Step 3: Let $a=0, b=1, c=0, i=1$

Step 4: for ($i=1; i \leq n; i++$)

Step 5: $a=b$

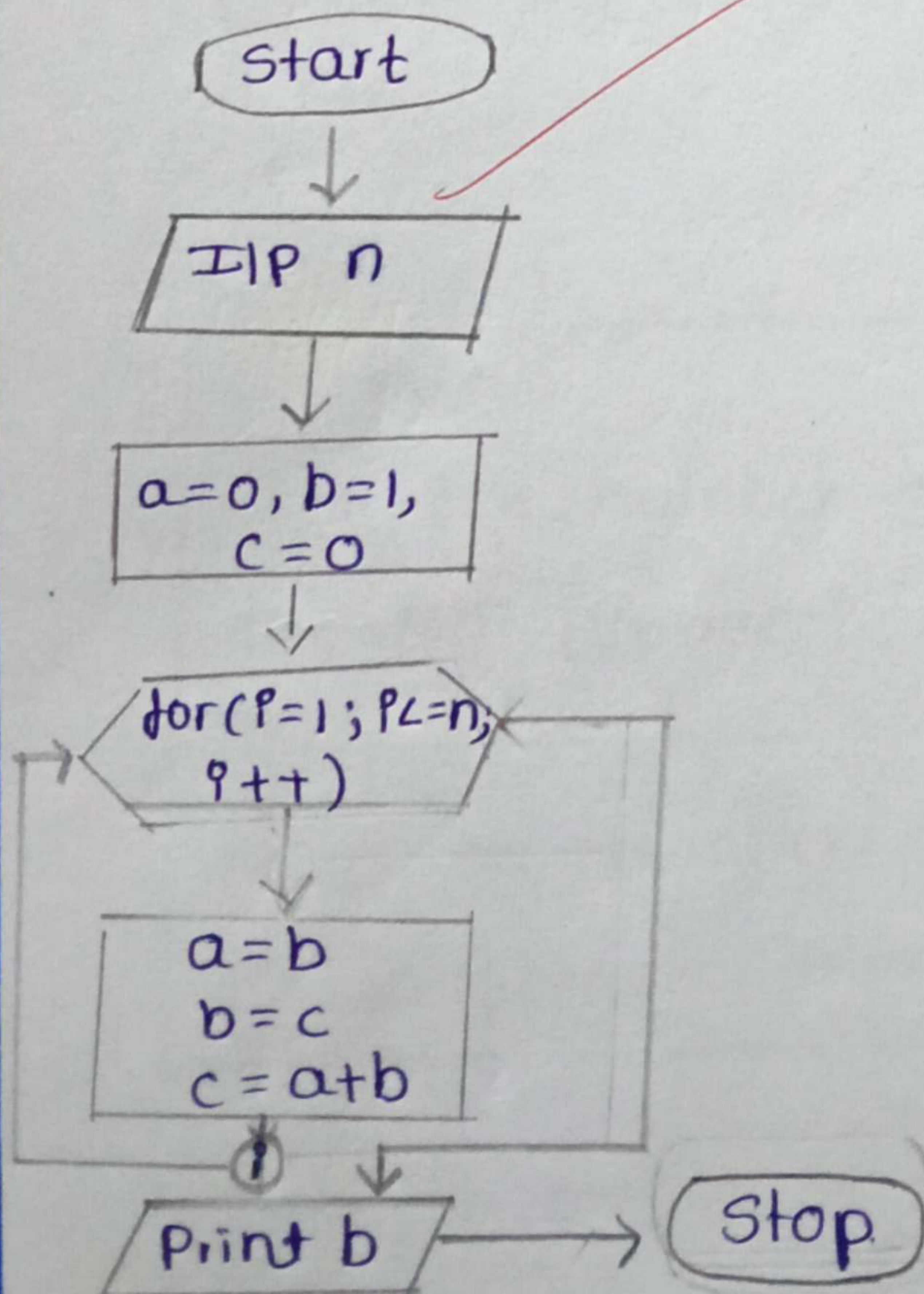
$b=c$

$c=a+b$

Step 6: Print b

Step 7: Stop.

Flowchart:



② Program to generate n^{th} fibonacci number

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int n, a=0, b=1, c=0, i=1;
    printf("enter the number \n");
    scanf("%d", &n);
    printf("The fibonacci number is : ", n);
    for(i=1; i<=n; i++)
    {
        a=b;
        b=c;
        c=a+b;
    }
    printf("%d\t", b);
    return 0;
}
```

fibonacci series
printf("%d", a);
c = a + b
a = b
b = c

output:-

enter the number 5

The 5th fibonacci number is 3

enter the number 3

The 3rd fibonacci number is 1

Lab Program - 02

② Write a C program to reverse the given decimal number

Algorithm

Step 1 : Start

Step 2 : Input n

Step 3 : let $rev = 0$, rem

Step 4 : while ($n > 0$)

Step 5 : $rem = n \% 10$

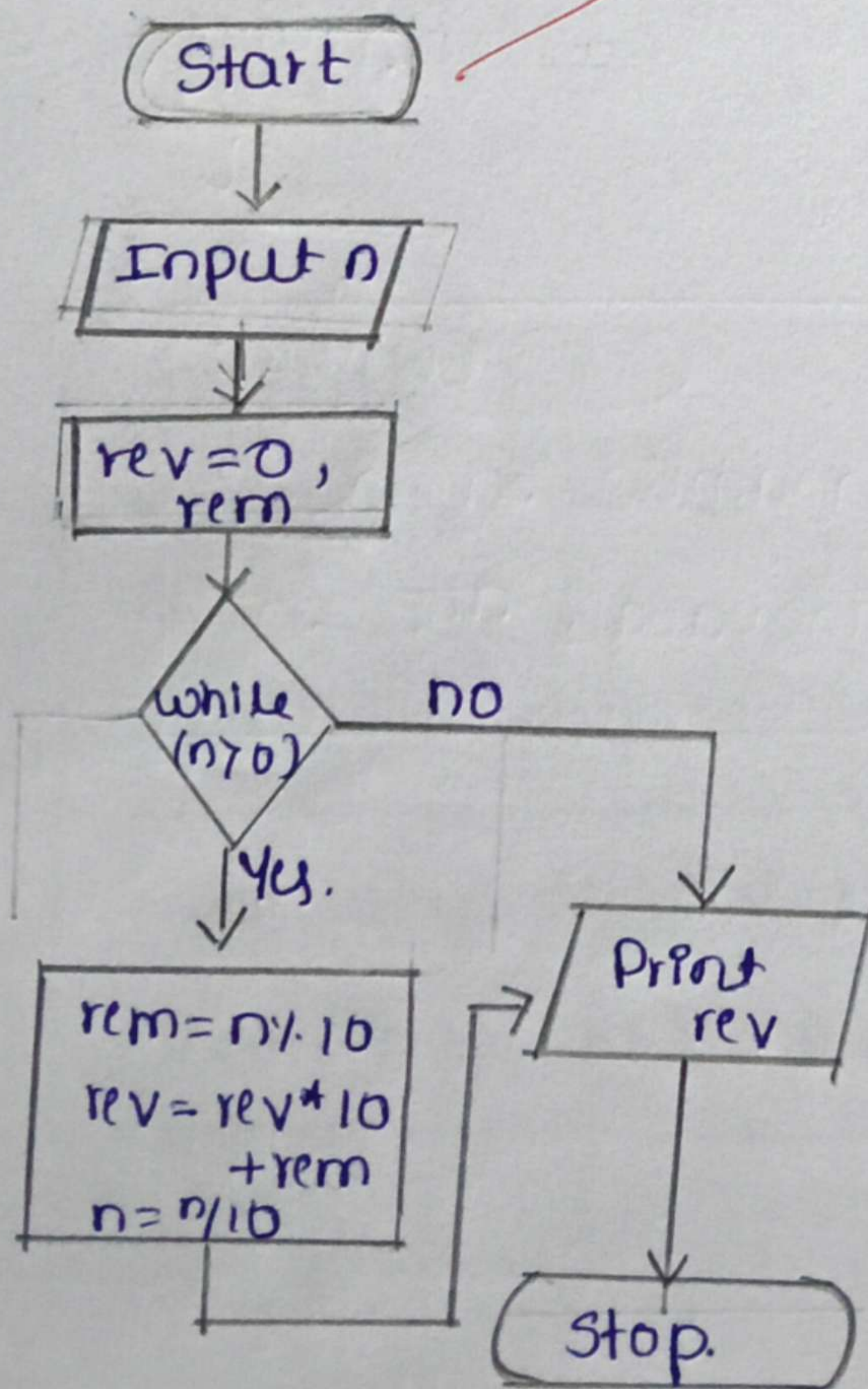
$rev = rev * 10 + rem$

$n = n / 10$

Step 6 : Print rev

Step 7 : Stop.

Flowchart.



② Program to reverse the given decimal number

```
#include <stdio.h>
#include <conio.h>
void main( )
{
    int n, rev = 0, rem;
    printf("enter any decimal numbers/n");
    scanf("%d", &n);
    while (n > 0)
    {
        rem = n % 10;
        rev = rev * 10 + rem;
        n = n / 10;
    }
    printf("The reverse of a given number is: %d", rev);
    getch();
}
```

output

enter any ^{decimal} number : 1234

The reverse of a
given number is : 4321

enter any decimal number 5213

The reverse of a given number is 3125

Lab Program -03

③ Write a C program to convert binary to decimal number

Algorithm:

Step 1: Start

Step 2: Input n

Step 3: $i=0$, rem , $sum=0$

Step 4: while($n > 0$)

Step 5: $rem = n \% 10$

$sum = sum + rem * Pow(2, i)$

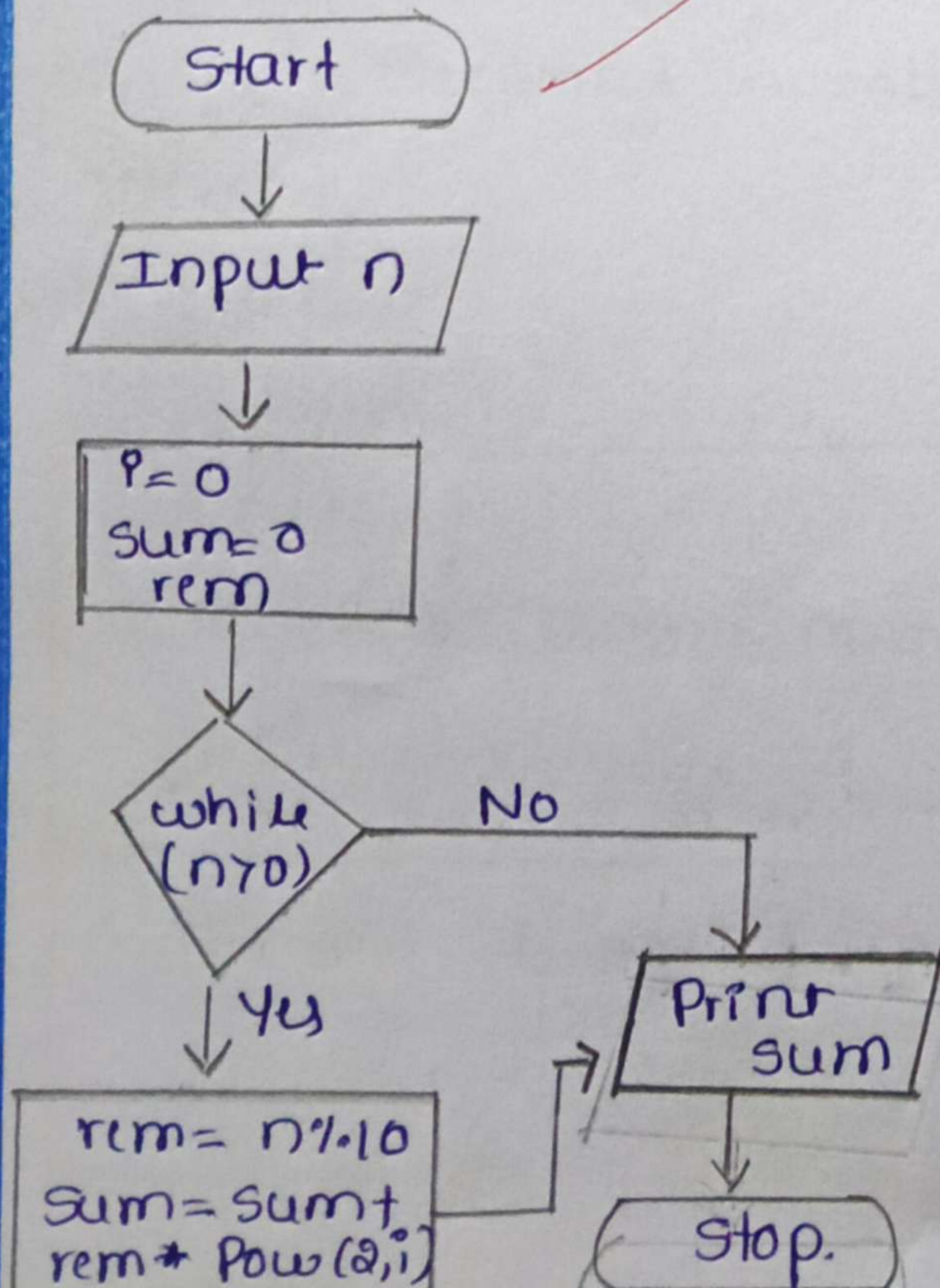
$n = n / 10$

$i++$

Step 6: Print sum

Step 7: stop

Flowchart:



③ C program to convert binary number to decimal number.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    long int n, rem, sum = 0, i = 0;
    printf("enter any binary number \n");
    scanf("%d", &n);
    while (n > 0)
    {
        rem = n % 10;
        sum = sum + rem * (pow(2, i));
        n = n / 10;
        i++;
    }
    printf("Decimal number is %d", sum);
    getch();
}
```

output:-

Enter any binary number 101101
Decimal number is 45

enter any binary number 1100
Decimal number is 12

Lab Program-04

④ Write a C program to calculate $\sin(x)$ value without using inbuilt function

Algorithm:

Step 1: Start

Step 2: Input n, x, sum

Step 3: Initialize $\text{sign}=1, \text{fact}=1$

Step 4: for ($i=3; i \leq n; i++$)

Step 5: $\text{fact} = \text{fact} * i * (i-1)$

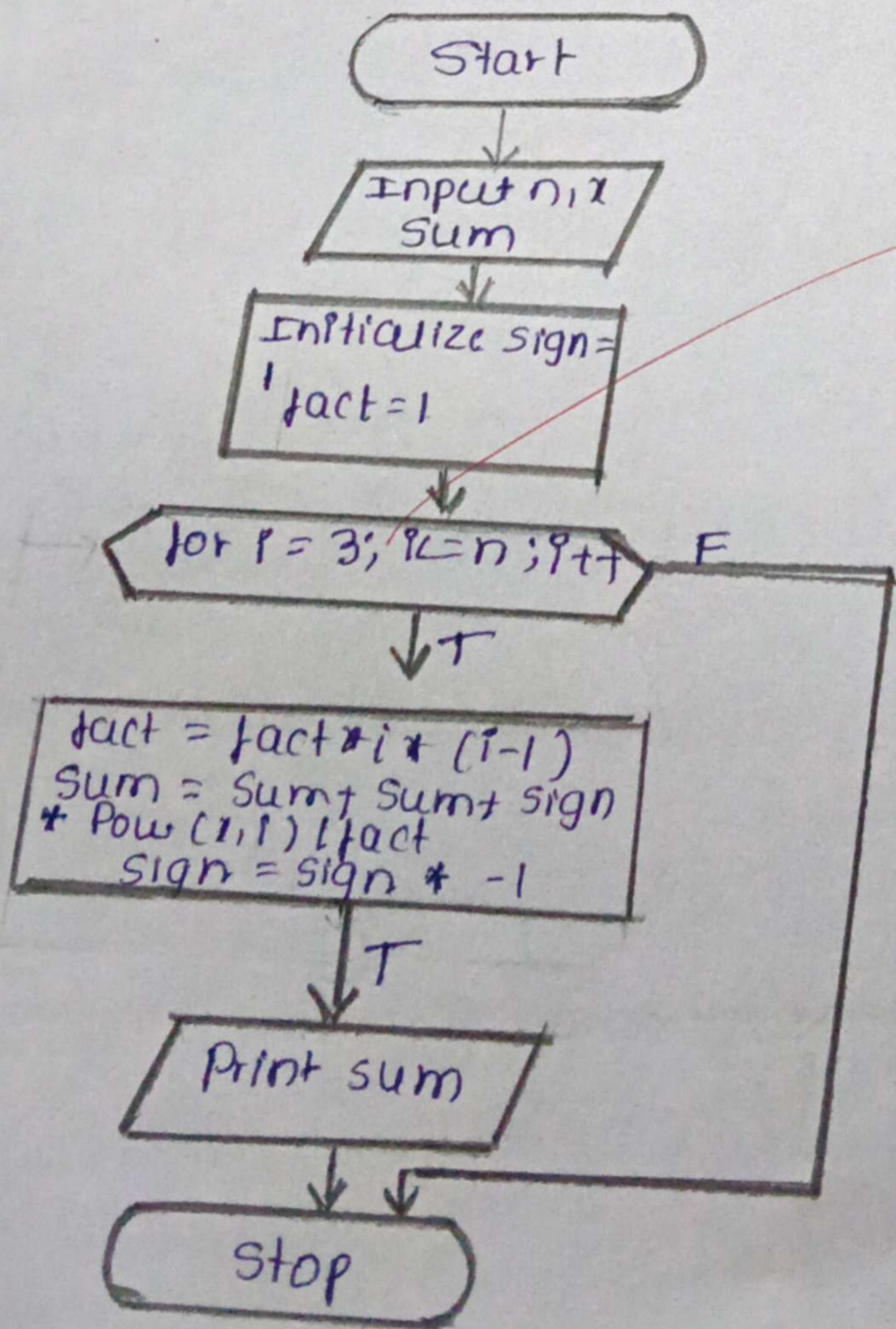
$\text{sum} = \text{sum} + \text{sum} + \text{sign} * \text{pow}(x, i) / \text{fact}$

$\text{sign} = \text{sign} * -1$

Step 6: Print sum

Step 7: stop

Flowchart



Lab Program-04

④ write a c program to calculate $\sin(x)$ value without using inbuilt function

Program:

```
#include <stdio.h>
#include <math.h>
int main()
{
    int n, fact = 1, sign = 1;
    float sum, x;
    printf("... Series calculator...\n\n");
    printf("Enter value of n:");
    scanf("%d", &n);
    printf("Angle in degree");
    scanf("%f", &x);
    x = x * 3.14 / 180;
    sum = x;
    printf("Angle in radian = %f", x);
    for (i = 3; i <= n; i++)
    {
        fact = fact * i * (i - 1);
        sum = sum + sum + sign * pow(x, i) / fact;
        sign = sign * -1;
    }
    printf("\n\n sum = %.4f", sum);
    printf("\n\n");
    return 0;
}
```

output

```
.... Series calculator...
Enter the value of n: 10
Angle in degree : 45
Angle in radian: 0.785000
Sum = 0.7068.
```


Lab Program -05

Q5) Write a C program to calculate $\cos(x)$ value without using inbuilt function

Algorithm:

Step 1: Start

Step 2: Input Angle - Degree

Step 3: Initialize $ans = 1$
 $temp = 1$, $acc = 3$

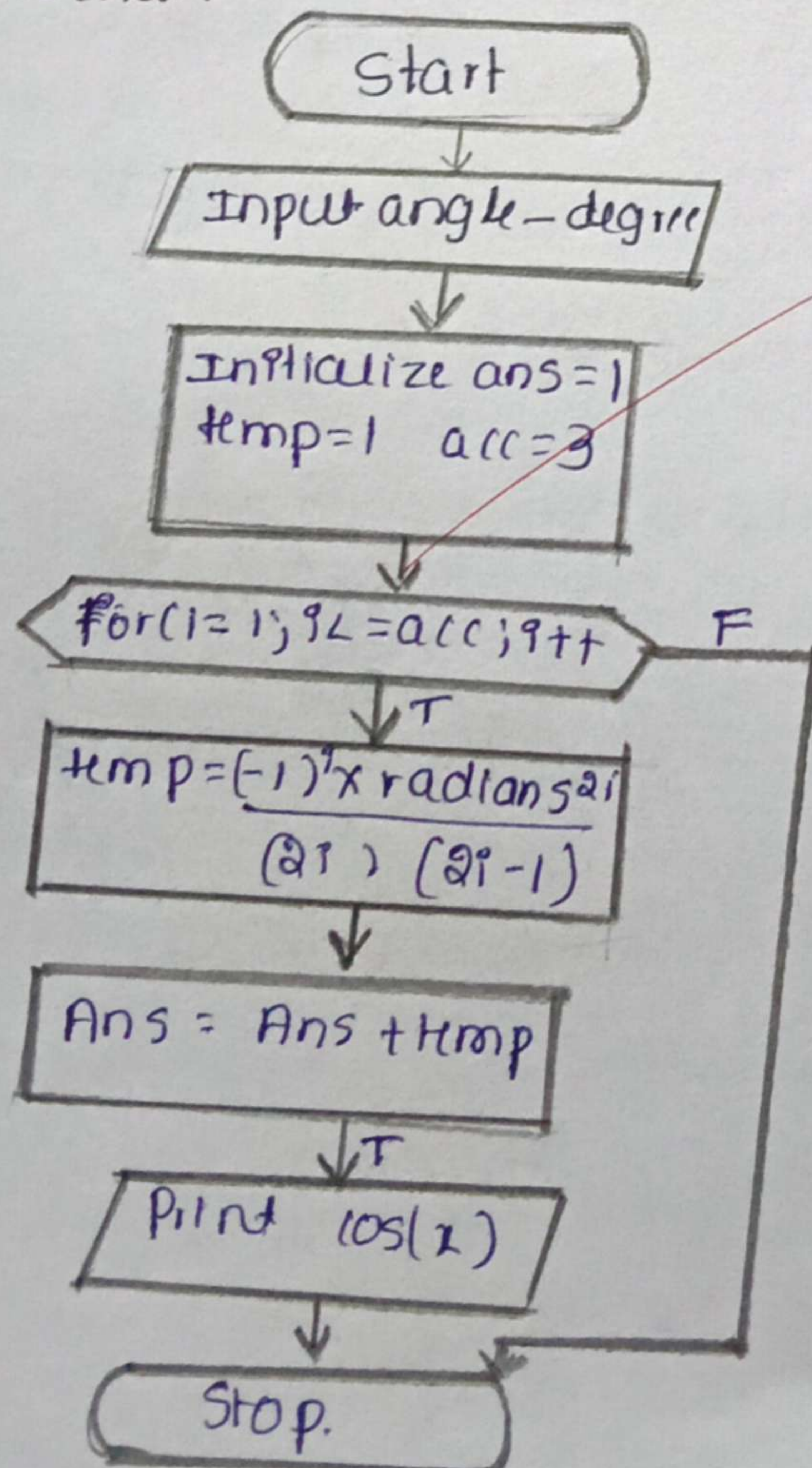
Step 4: for ($i = 1$; $i \leq acc$; $i++$)

Step 5: calculation $temp = (-1)^i \times \text{radians}^{2i}$
 $(2i) (2i-1)$
 $ans = ans + temp$

Step 6: Print $\cos x$

Step 7: Stop

Flowchart



Lab Program -05

(05) Write a C program to calculate $\cos(x)$ value without using inbuilt function

Program:

```
#include <stdio.h>
#include <conio.h>
#define Pi 3.14
void main()
{
    printf("Enter the angle in degree\n");
    float angle-degree;
    scanf("%f", &angle-degree);
    float angle-radian = angle-degree * Pi/180;
    float ans = 1;
    float temp = 1;
    int acc = 3;
    int i;
    for (i = 1; i <= acc; i++)
    {
        temp = temp * (-1) * angle-radian * angle-radian / (i * (i+1));
        ans = ans + temp;
    }
    printf("cos(x) = %f\n", angle-degree, ans);
    getch();
}
```

Output:

Enter angle in degree

90

-0.44

17-10-20

Lab Program - 06

⑥ Write a C program to calculate e^x value without using inbuilt functions: Algorithm

Step 1: Start

Step 2: Input n, x

Step 3: Initialize $Ans = 1$
 $temp = 1$ $acc = 30$

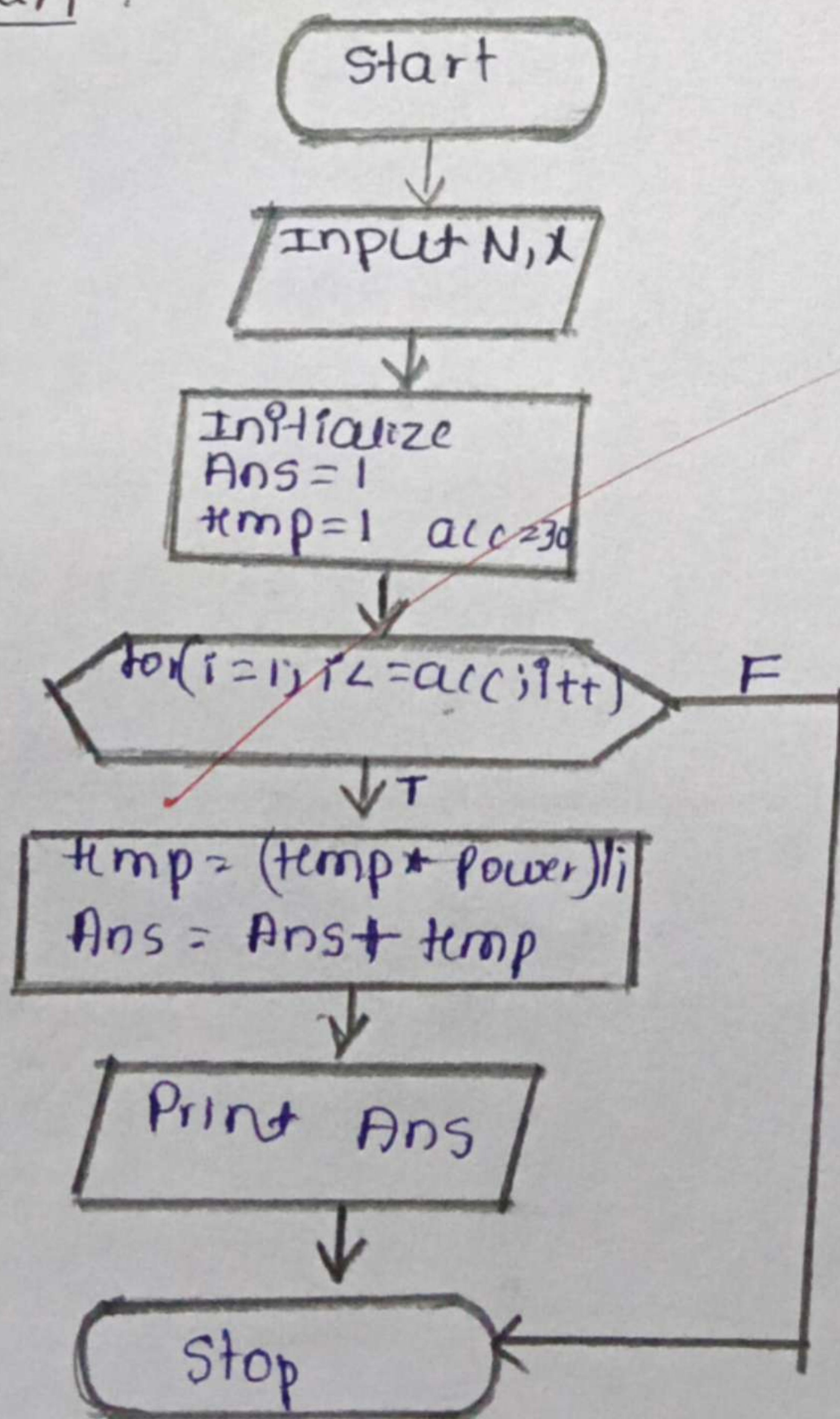
Step 4: for ($i = 1; i \leq acc; i++$)

Step 5: calculate $temp = \frac{temp}{i} \times x$
 $Ans = Ans + temp$

Step 6: Print e^x

Step 7: Stop

Flowchart:



Lab Program - 06.

⑥ Write a C program to calculate e^x value without using inbuilt functions

Program:

```
#include <stdio.h>
#include <conio.h>
void main()
{
    float power;
    printf("Enter the power of e\n");
    scanf("%f", &power);
    int acc = 30;
    float ans = 1;
    float temp = 1;
    int i;
    for (i = 1; i < acc; i++)
    {
        temp = (temp * power) / i;
        ans = ans + temp;
    }
    printf("%f", ans);
}
```

Output

Enter the value of power of e
5

148.413177

Lab Program - 07

⑦ Write a C program to calculate gcd of two numbers

Algorithm

Step 1: Start

Step 2: Input n_1, n_2

Step 3: If $(n_1 == n_2)$ go to step 5

Step 4: If $(n_1 > n_2)$

$n_1 = n_1 - n_2$

go to step 3

else

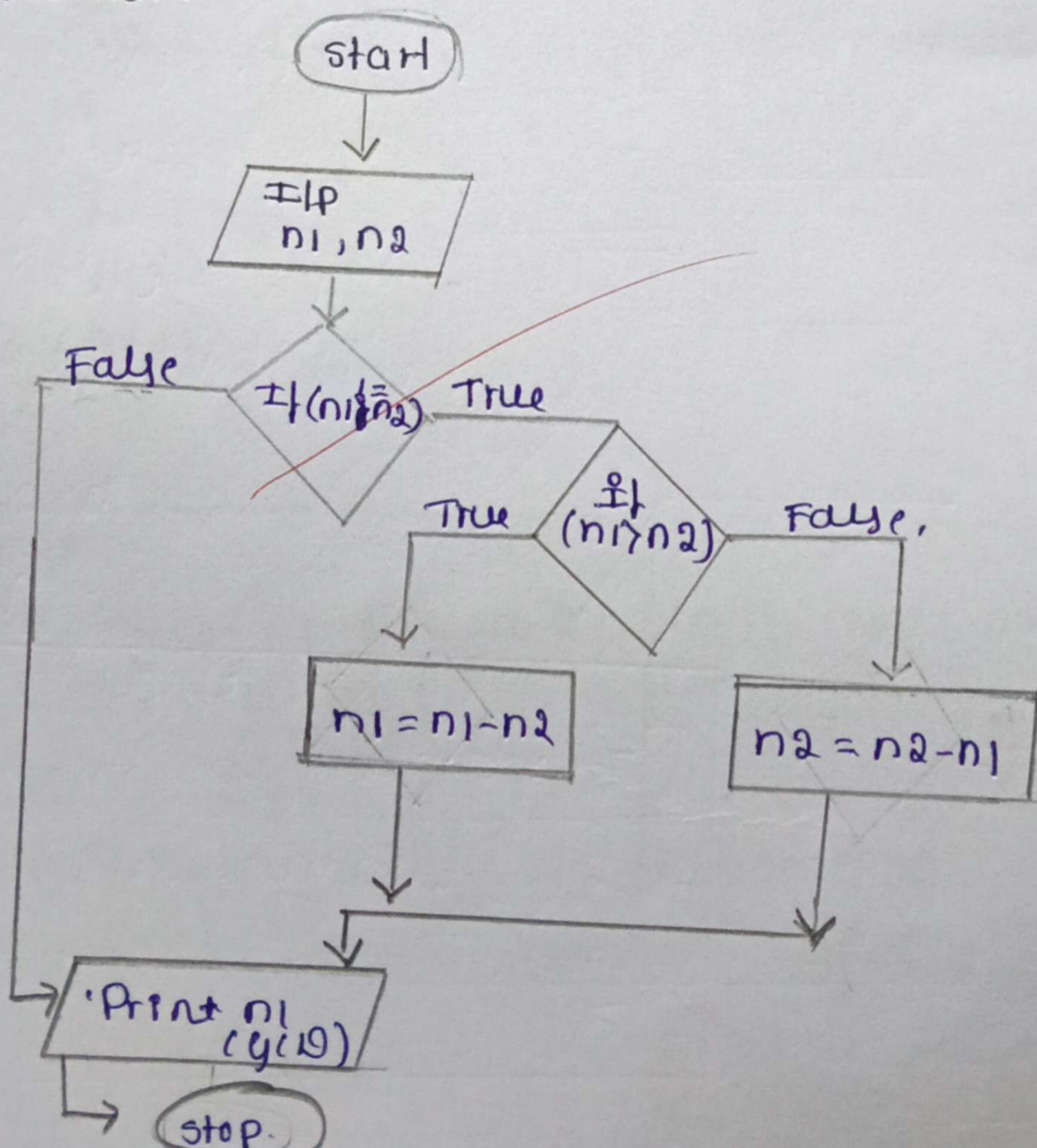
$n_2 = n_2 - n_1$

Step 5: Print n_1

go to step 3

Step 6: stop

Flowchart



Program to calculate gcd of two numbers

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n1, n2;
    printf("enter n1 and n2 \n");
    scanf("%d %d", &n1, &n2);
    while (n1 != n2)
    {
        if (n1 > n2)
        {
            n1 = n1 - n2;
        }
        else
        {
            n2 = n2 - n1;
        }
    }
    printf("gcd = %d", n1);
    getch();
}
```

output

Enter num1 and num2
(n1 and n2)
10 20
gcd = 10

enter num1 and num2
45 25
gcd = 5

Lab Program - 08

⑧ Write a C program to generate prime numbers

Algorithm:

Step 1: Start

Step 2: Input n

Step 3: Let $c = 0$ (count = 0)

Step 4: for ($i = 2; i \leq n; i++$)

Step 5: $c = 0$

for ($j = 1; j \leq i; j++$)

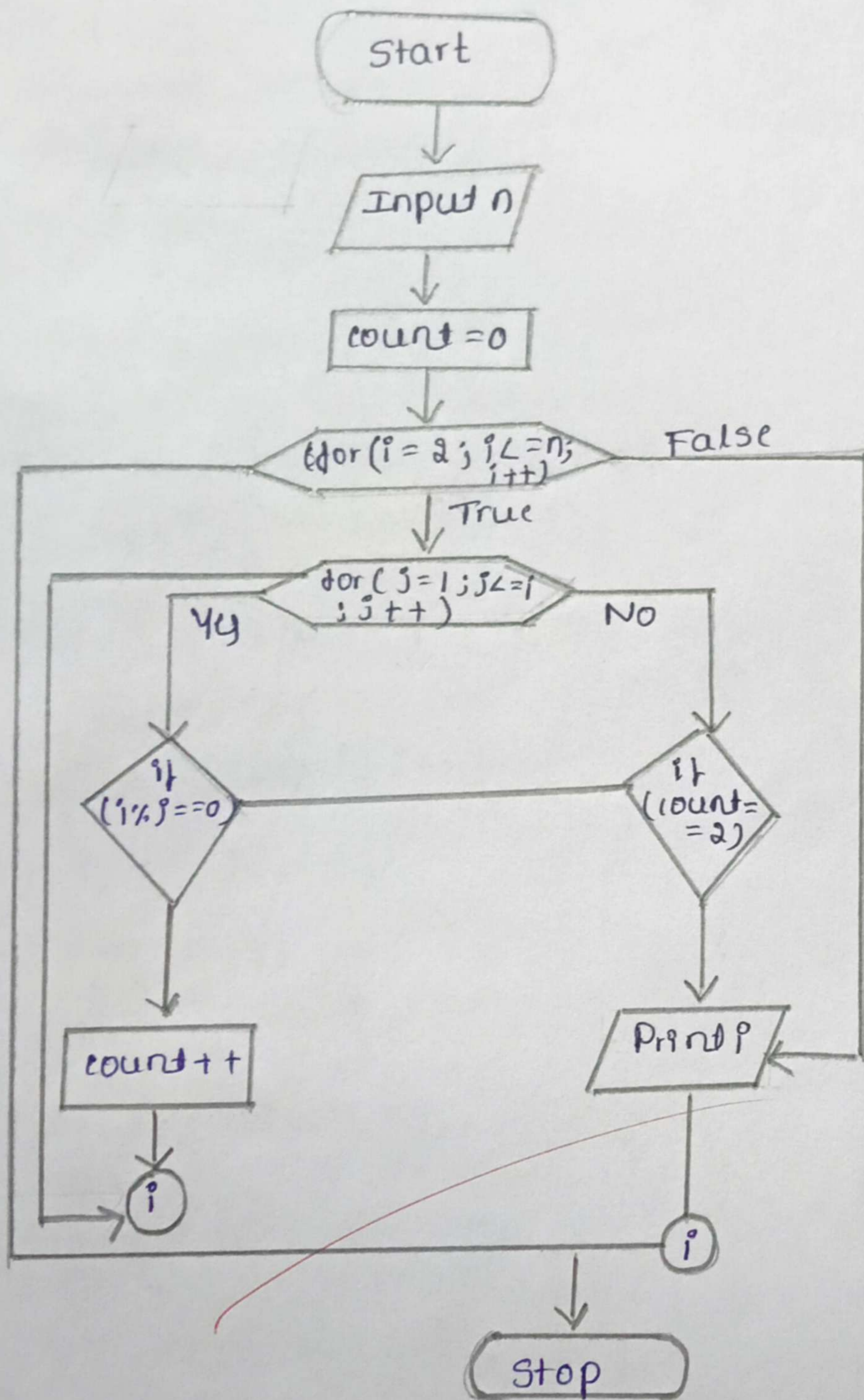
if ($i \% j == 0$)

$c++$

Step 6: if (count == 2)

Step 7: Print i

Step 8: Stop.



⑧ Program to generate prime numbers

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n, count = 0, i, j;
    printf("enter any number\n");
    scanf("%d", &n);
    printf("Prime numbers");
    for (i = 2; i <= n; i++)
    {
        count = 0;
        for (j = 1; j <= i; j++)
        {
            if (i % j == 0)
                count++;
        }
        if (count == 2)
            printf("%d", i);
        getch();
    }
```

output

enter any number 5

Prime numbers 2

3

5

enter any number 11

2

3

5

7

11

Scanned by
17-10-24

Lab Program - 09

⑨ Write a C program to find square root of a number without inbuilt function

Algorithm :

Step 1 : Start

Step 2 : Input N

Step 3 : Let temp = 0, sqrt

Step 4 : $\text{sqrt} = \text{number} / 2$

Step 5 : while ($\text{sqrt} \neq \text{temp}$)

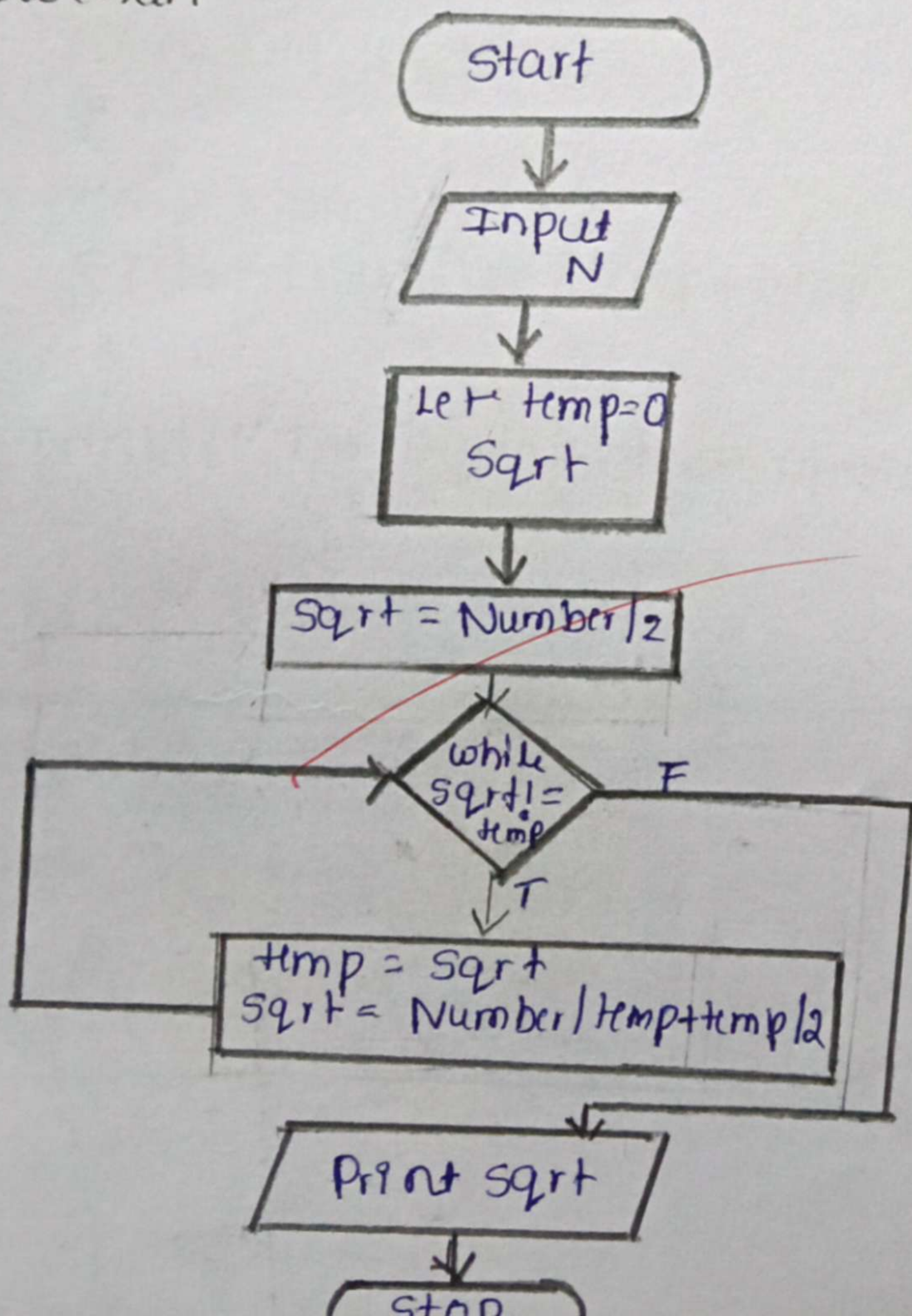
Step 6 : $\text{temp} = \text{sqrt}$

$\text{sqrt} = (\text{Number} / \text{temp} + \text{temp}) / 2$

Step 7 : Print sqrt

Step 8 : Stop

Flowchart



Lab Program-09

Q) Write a C program to find square root of a number without inbuilt function.

Program:

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int number;
    float temp, sqrt;
    printf("Enter the number: \n");
    scanf("%d", &number);
    sqrt = number / 2;
    temp = 0;
    while (sqrt != temp)
    {
        temp = sqrt;
        sqrt = (number / temp + temp) / 2;
    }
    printf("The square root of %d is %f", number, sqrt);
}
```

Output

Enter a number : 25

The square root of 25 is 5

Lab Program 10

(10) C program to convert character to ascii code

Algorithm

Step 1: Start

Step 2: Input ch.

Step 3: Convert character ch to its ascii code

Step 4: Display ascii code

Step 5: Stop

Program

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main ()
```

```
{
```

```
char ch;
```

```
printf ("enter any character \n");
```

```
scanf ("%c", &ch);
```

```
printf ("%d", ch);
```

```
getch();
```

```
}
```

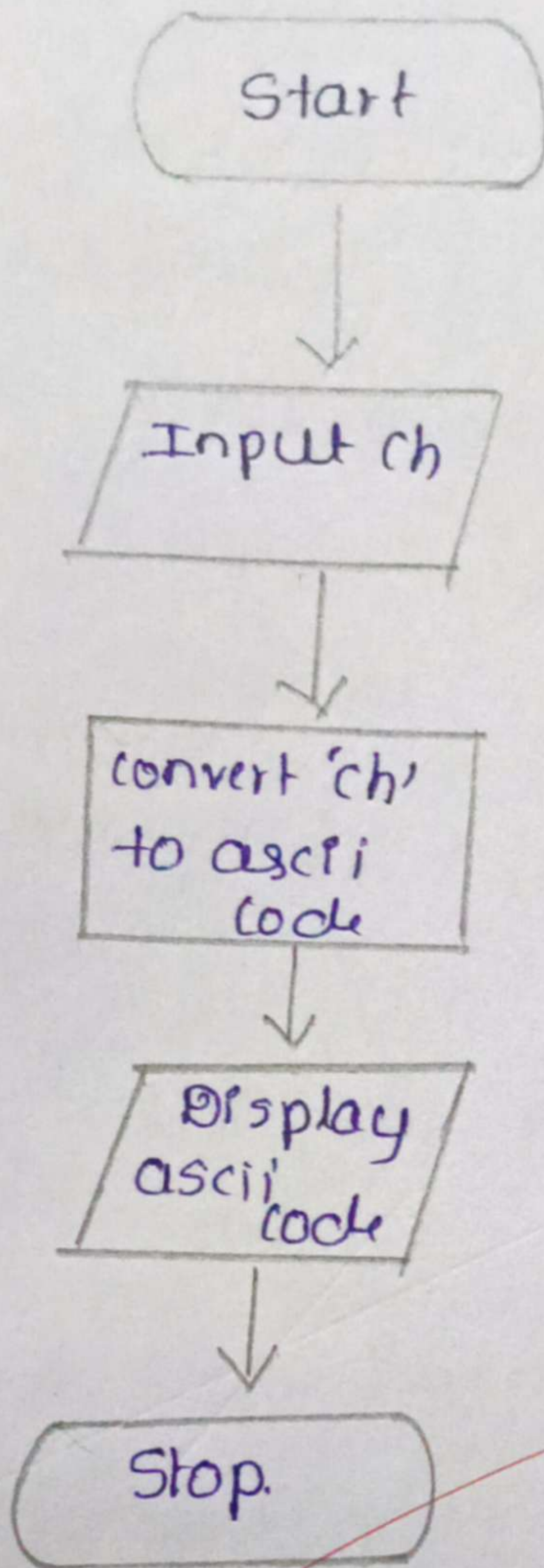
Output

enter any character A

65

Lab Program -10

Flowchart



Lab Program -11

C program for partition Array

Algorithm.

Step 1: Start

Step 2: Read n

Step 3: for $i=0$ to $n-1$

 Read $A[i]$

end for

Step 4: $mid = n/2$

Step 5: for $i=0$ to mid

 Display $A[i]$

end for

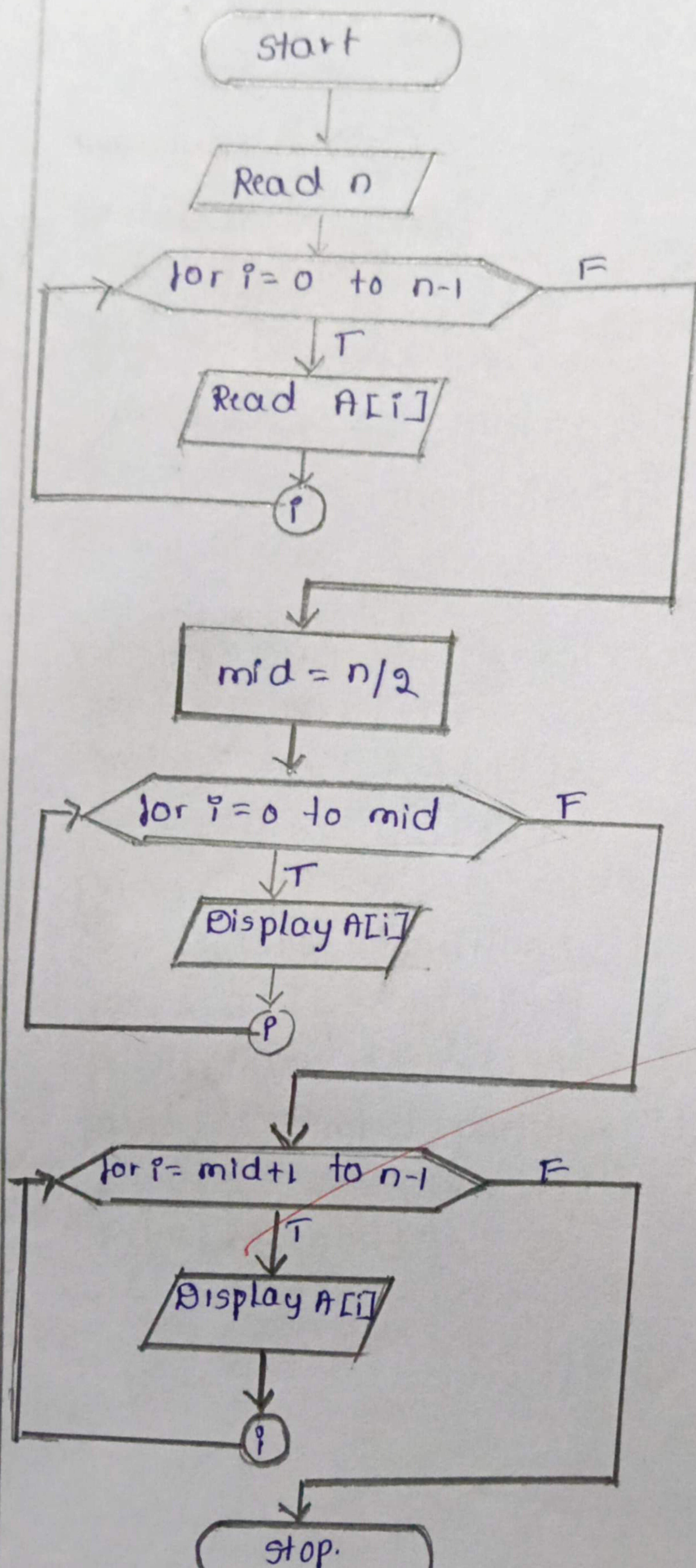
Step 6: for $i = mid+1$ to $n-1$

 Display $A[i]$

end for

Step 7: Stop

Flowchart



Lab Program - II

(I) Write a C program to partition of an array

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[10], n, i, mid = 0;
    printf("Enter the limit:");
    scanf("%d", &n);

    printf("Enter %d elements", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    mid = (0 + (n - 1)) / 2;
    printf("Mid = %d \n", mid);
    printf("First Partition:");
    for (i = 0; i <= mid; i++)
        printf("\n%d", a[i]);
    printf("Second partition:");
    for (i = mid + 1; i < n; i++)
        printf("\n%d", a[i]);
}
```


Output

Enter the limit: 5

Enter the 5 elements: 11

22

33

44

55

Entered numbers are: 11

22

33

44

55

First partition is: 11

22

33

Second partition is: 44

55

Geo
17-19-04

Lab Program - 12

⑫ C program to remove the duplicates from ordered array

Algorithm:

Step 1 : Start

Step 2 : Read n

Step 3 : for $i = 0$ to n

 Read $A[i]$

 end for

Step 4 : for $i = 0$ to n

 for $j = i + 1$ to n

 if $(A[i] == A[j])$ then

 for $k = j$ to $n - 1$

$A[k] = A[k + 1]$

 end for

$n = n - 1$

$j = j - 1$

 end if

 end for

 end for

Step 5 : for $i = 0$ to n

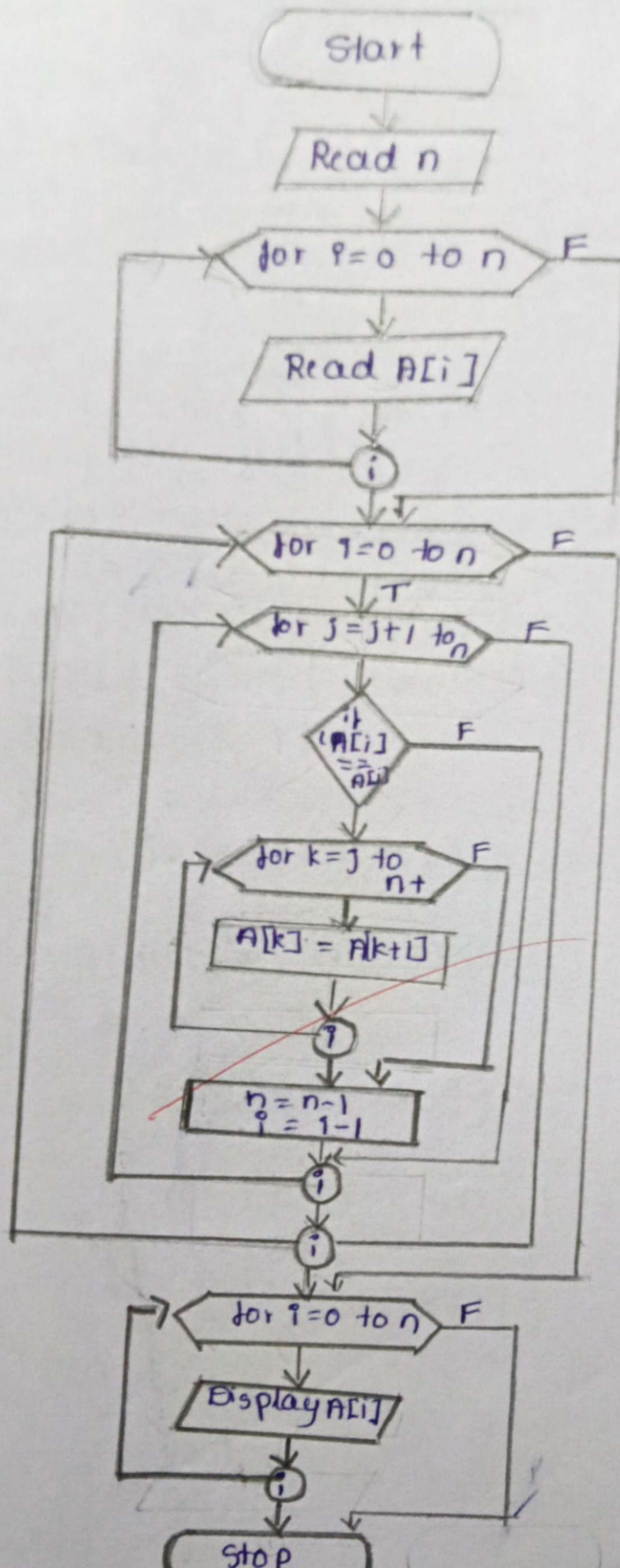
 Display $A[i]$

 end for

Step 6 : stop.

Lab Program -12

Flowchart



Lab Program - 12

⑫ Write a C program to remove the duplicates from the given array.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[10], n, i, j, k;
    printf("Enter the limit:");
    scanf("%d", &n);
    printf("Enter %d numbers:", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Entered numbers are:");
    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
if (a[i] == a[j])
            {
if (a[i] == a[j])
                for (k = j; k < n; k++)
                {
                    a[k] = a[k + 1];
                }
                n--;
                j--;
            }
        }
    }
```


}

}

}

printf("After removal of duplicates:");

for (i=0; i<n; i++)

printf("%d\n", a[i]);

getch();

}

Output

Enter the limit: 5

enter 5 numbers: 10

11

11

12

13

Entered numbers are

10

11

11

12

13

After removal of duplicaty

10

11

12

13

Enter the limit: 3

enter 3 numbers: 55

56

55

Entered numbers are: 55

56

55

After removal of
duplicatu

55

56

Lab Program -13

(13) Count duplicates : Algorithm

Step 1: Start

Step 2: Read n

Step 3: for $i = 0$ to n
Input $A[i]$

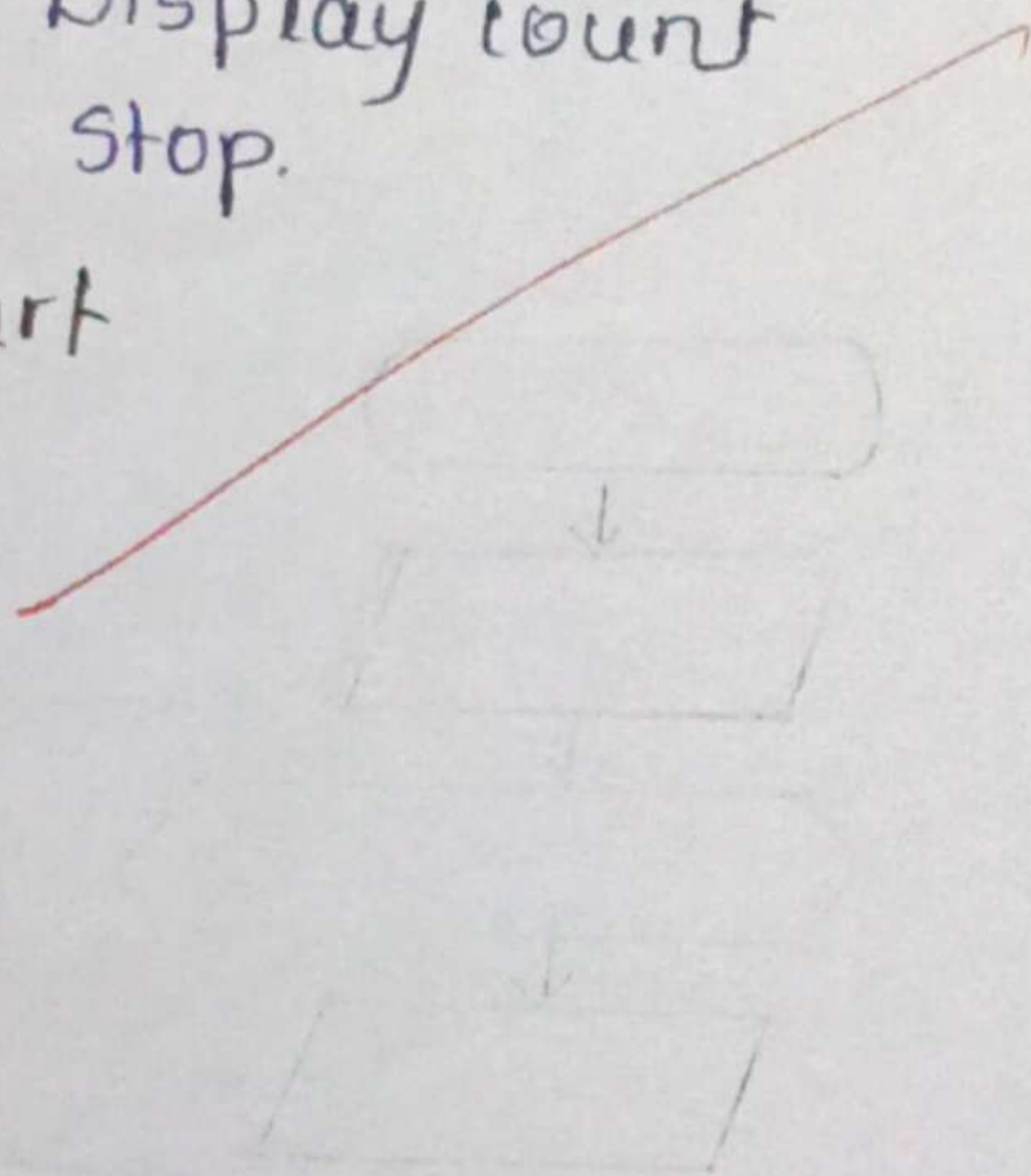
Step 4: Output $A[i]$

Step 5: for $i = 0$ to n
for $j = i+1$ to n
if $A[i] = A[j]$
count = count + 1
break;

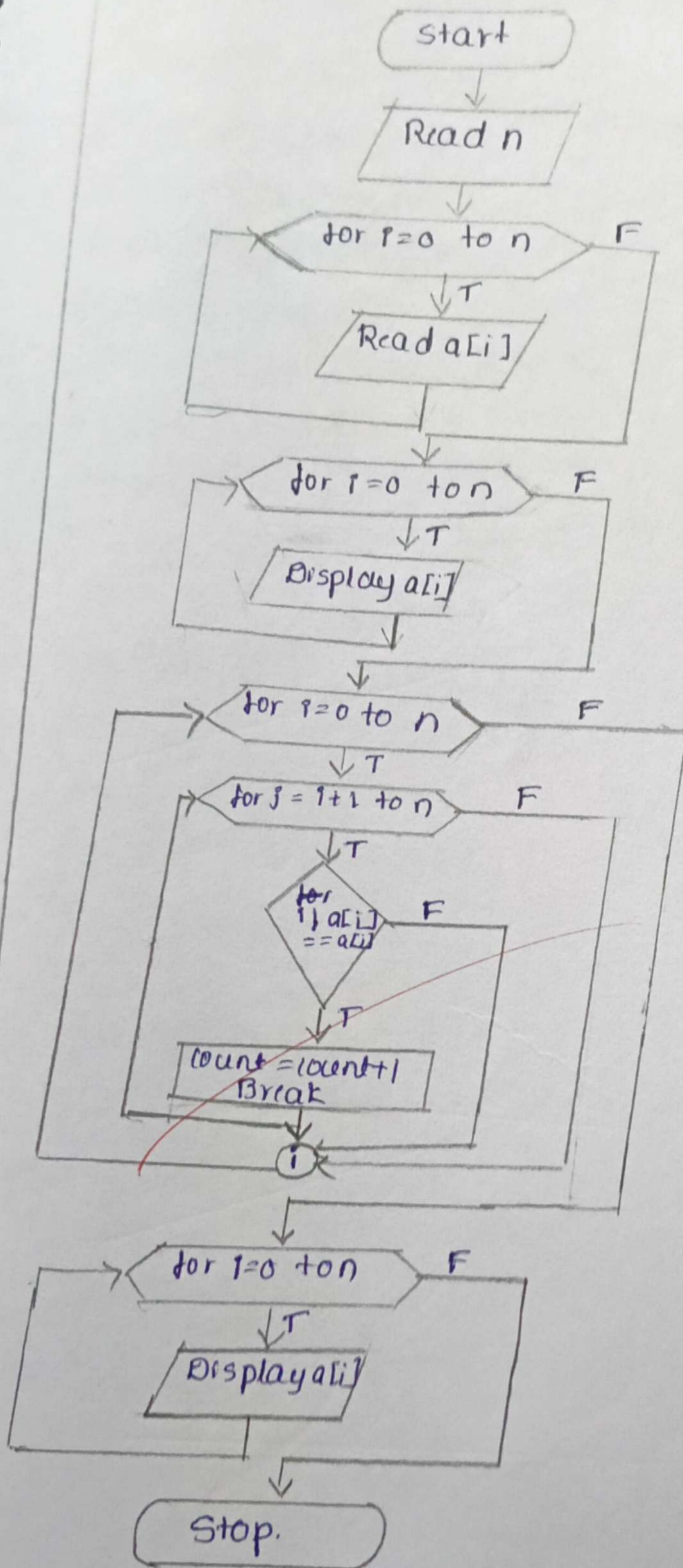
Step 6: Display count

Step 7: Stop.

Flowchart



Flowchart :



Lab Program -13

⑬ Write a C program to count duplicate elements from an Array.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[10], n, i, j, count=0;
    printf("Enter the limit:");
    scanf("%d", &n);
    printf("Enter %d elements:", n);
    for (i=0; i<n; i++)
        scanf("%d", &a[i]);
    printf("Entered numbers are:");
    for (i=0; i<n; i++)
        printf("%d\n", a[i]);
    for (i=0; i<n; i++)
    {
        for (j=i+1; j<n; j++)
        {
            if (a[i] == a[j])
            {
                count++;
                break;
            }
        }
    }
}
```



```
printf("\n COUNT = %.d", count);  
getch();  
}
```

Output

Enter the limit 5

Enter 5 elements 10

20

10

30

20

Entered numbers are 10

20

10

30

20

COUNT = 2

Sing
17-10-24

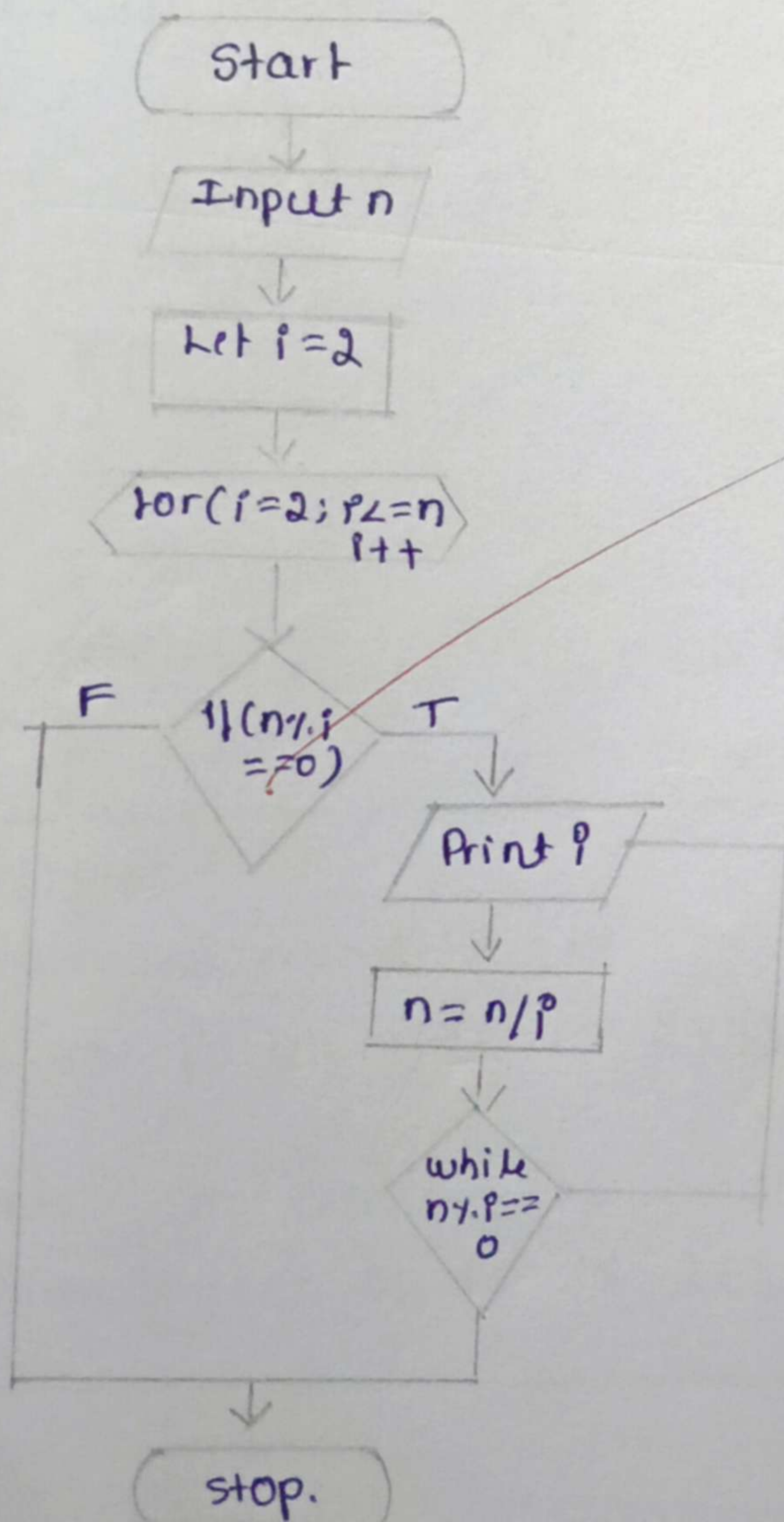
Lab Program -14

⑭ Write a C program to calculate prime factors of a number.

Algorithm

- Step 1: Start
- Step 2: Input n, p
- Step 3: for($i=2; i \leq n; i++$)
- Step 4: If ($n \% i == 0$)
- Step 5: while($n \% i == 0$)
- Step 6: Print i
 $n = n / i$
- Step 7: Stop.

Flowchart



Program:

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n, i;
    printf("enter a number \n");
    scanf("%d", &n);
    for (i = 2; i <= n; i++)
    {
        if (n % i == 0)
        {
            while (n % i == 0)
            {
                printf("%d", i);
                n = n / i;
            }
        }
    }
    getch();
}
```

output.

enter a number - 18

Prime factor of 18 is 2x3x3

enter a number 16

Prime factor of 16 is 2x2x2x2

Lab Program - 15

(15) Write a C program to reverse an array

Algorithm

Step 1 : Start

Step 2 : Input n

Step 3 : for $i = 0$ to n

 Read $A[i]$

end for

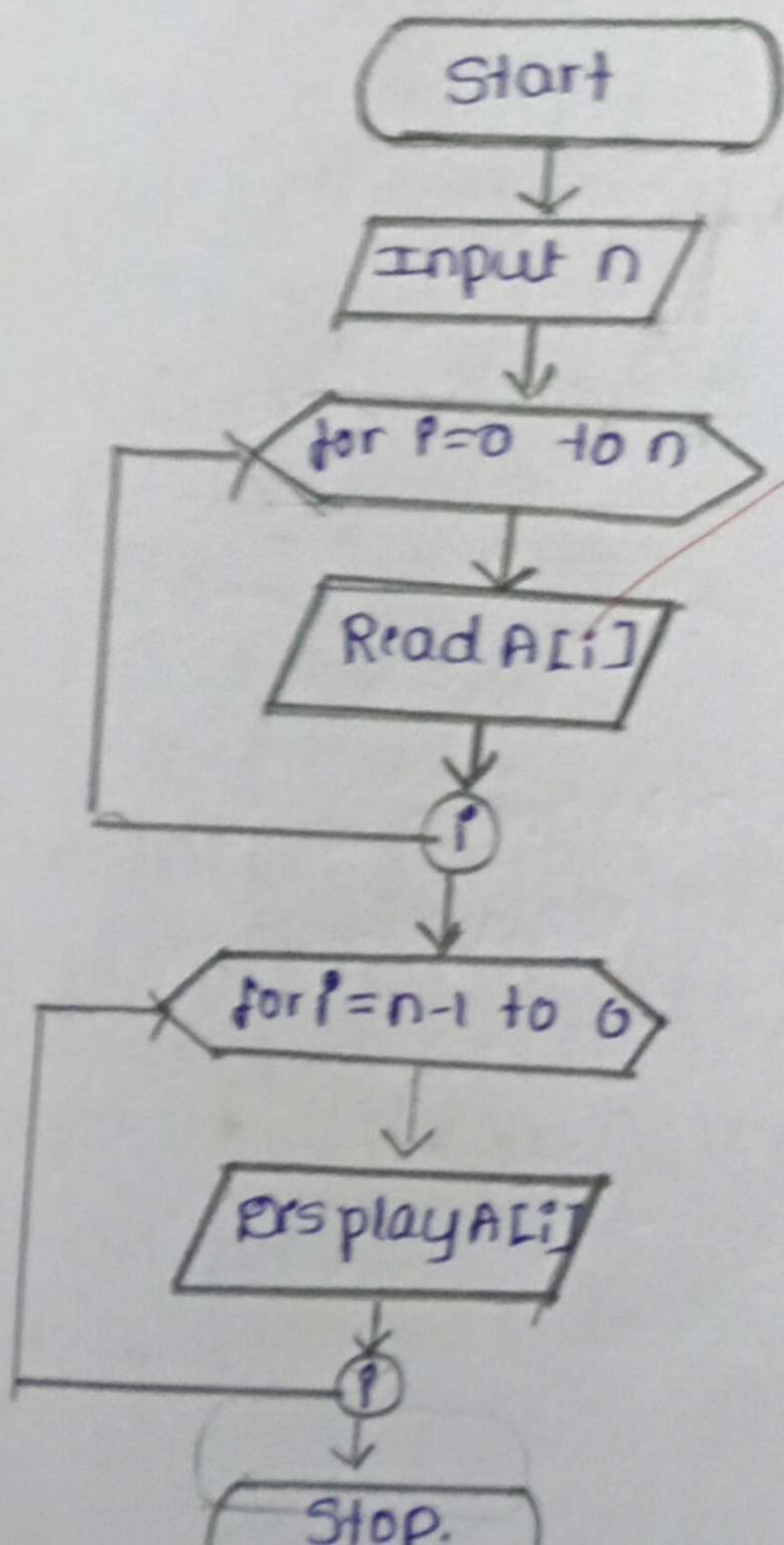
Step 4 : for $i = n-1$ to 0

 Display $A[i]$

end for

Step 5 : Stop.

Flowchart:



Program to reverse an Array :

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[10], rev[10], n, i, j;
    printf("Enter the limit:");
    scanf("%d", &n);

    printf("Enter %d elements:", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Entered array elements are:");
    for (i = 0; i < n; i++)
        printf("%d\n", a[i]);
    j = 0;
    for (i = n - 1; i >= 0; i--)
    {
        rev[j] = a[i];
        j++;
    }
    printf("Reversed array elements are:");
    for (i = 0; i < n; i++)
        printf("%d\n", rev[i]);
    getch();
}
```


Output

Enter the limit : 5

Enter the 5 elements : 10

20

30

40

50

Entered array elements are : 10

20

30

40

50

Reversed array elements are 50

40

30

20

10

Enter limit : 3

Enter 3 elements : 60

70

80

Entered elements are : ~~80~~ 60

70

~~80~~ 80

Reversed array elements 80

are 70

60.

Lab Program - 16

①6 Write a C program to calculate k^{th} smallest number in an array

Algorithm:

Step 1: Start

Step 2: Read n

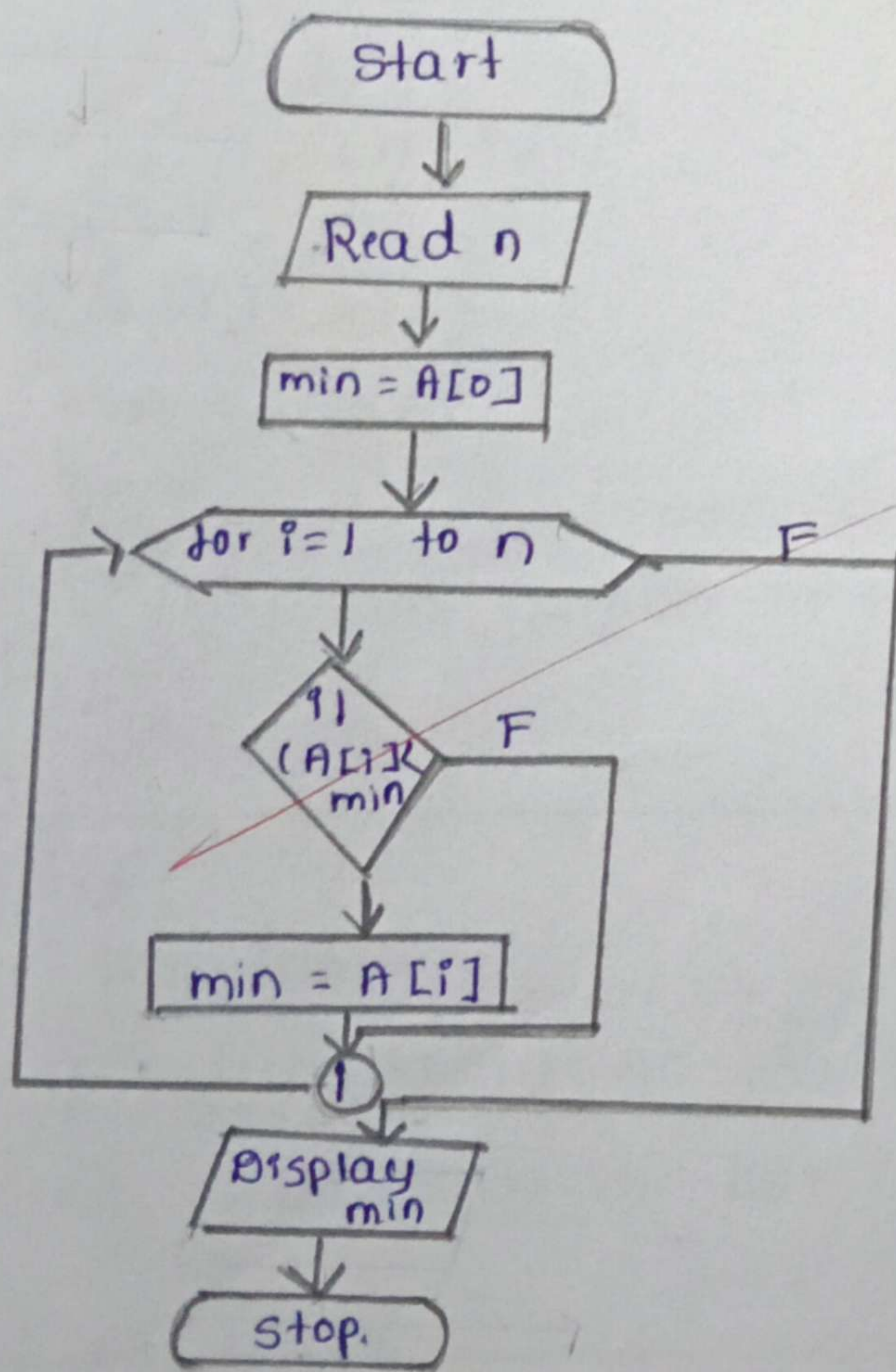
Step 3: $\text{min} = A[0]$

Step 4: for $i=1$ to n
 if $(A[i] < \text{min})$
 $\text{min} = A[i]$
 end if
end for

Step 5: Display min

Step 6: Stop

Flowchart



Lab Program - 16

⑥ Write a C program to calculate k^{th} smallest number in an array

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[10], n, i, min;
    printf("Enter the limit");
    scanf("%d", &n);

    printf("Enter %d numbers:", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    min = a[0];
    for (i = 0; i < n; i++)
    {
        if (a[i] < min)
            min = a[i];
    }

    printf("Minimum number in the list is %d", min);
    getch();
}
```

Output

Enter the limit 5

enter 5 elements 10 20 30 40 50

minimum number in the list is 10

Lab Program -17

(17) Write a C program to implement merge sort.

Algorithm:

Step 1: Start

Step 2: Divide the array. The array is divided into two halves

Step 3: Divide each half, continue dividing each half into smaller subarrays until each subarray has only one element

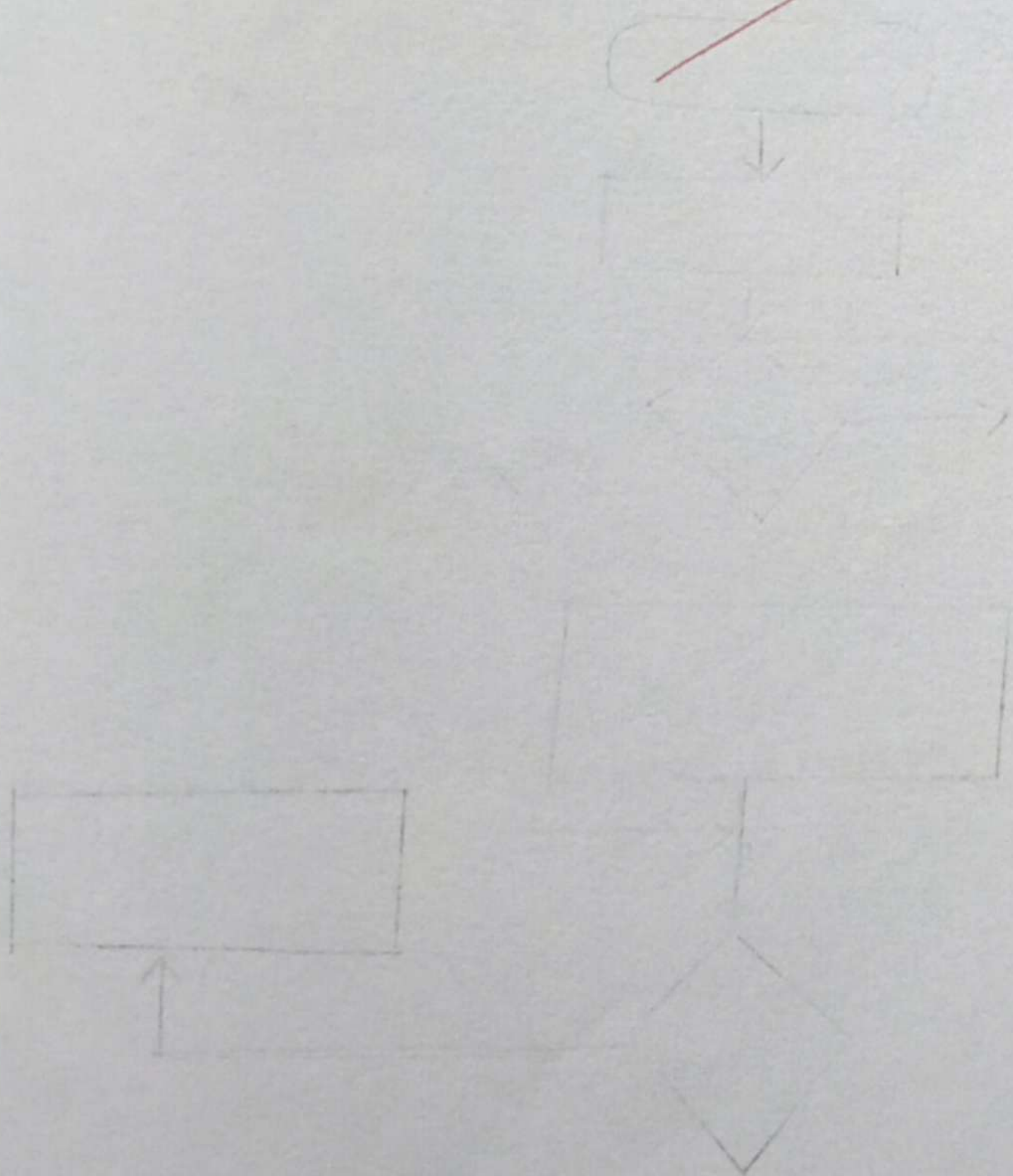
Step 4: Merge each pair of subarrays

Step 5: Merge the sorted subarrays

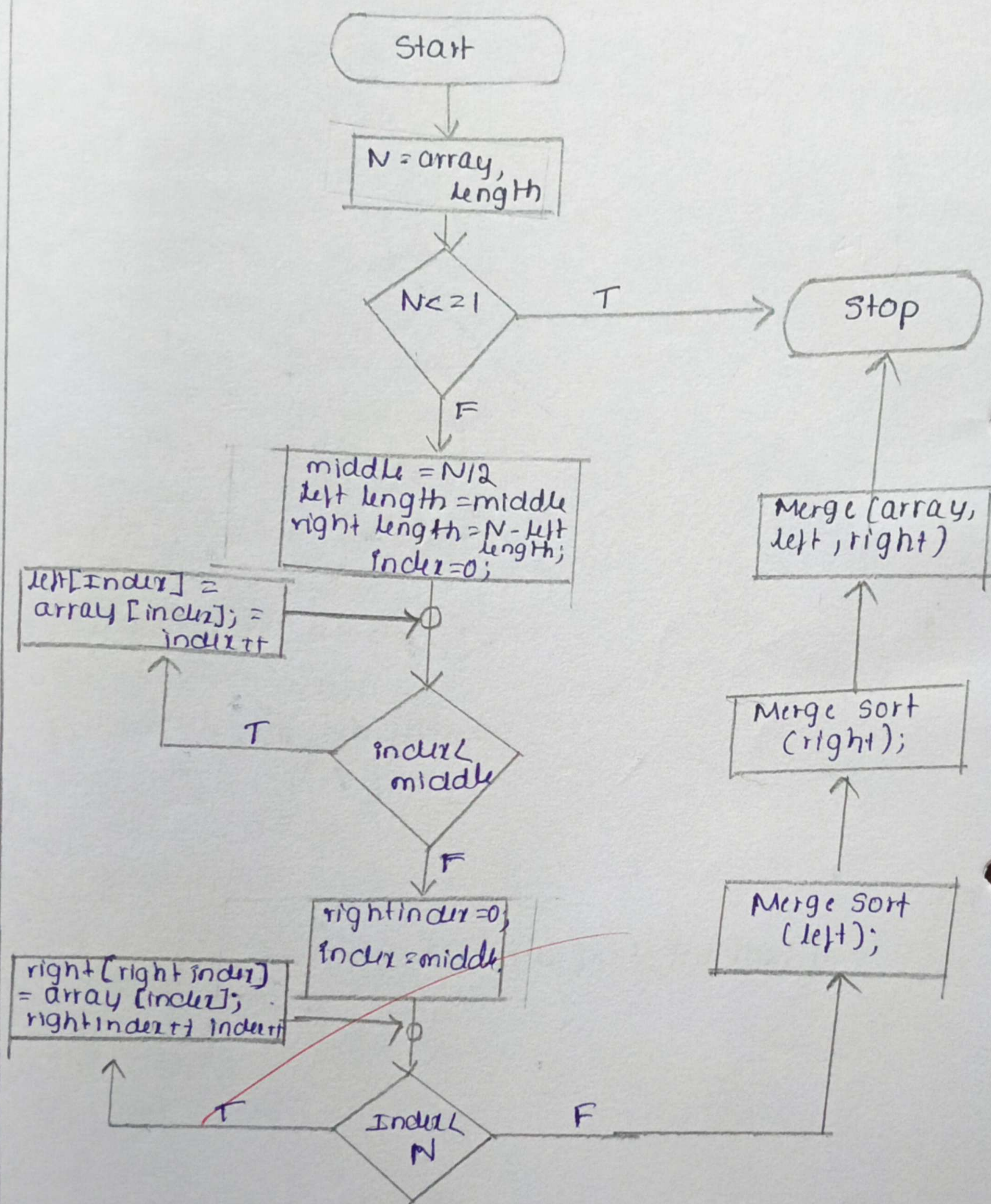
Step 6: Merge the final two halves

Step 7: Final sorted array

Step 8: Stop



Flowchart



Lab Program - 17

①7 Write a C program to implement merge sort

```
#include <stdio.h>
#include <conio.h>
void mergesort();
void merge();
void merge ();
void main()
{
    int i, a[50], n;
    printf("Enter the limit:");
    scanf ("%d", &n);
    printf("Enter the elements");
    for (i=0; i<n; i++)
        scanf ("%d", &a[i]);
    mergesort(a, 0, n-1);
    printf("Sorted elements are:");
    for (i=0; i<n; i++)
        printf ("%d", a[i]);
    getch();
}

void mergesort(int a[], int first, int last)
{
    int mid;
    if (first < last)
    {
        mid = (first + last) / 2;
        mergesort(a, first, mid);
        mergesort(a, mid + 1, last);
        merge (first, mid, last);
    }
}
```



```

void merge (int a[], int first, int mid, int last)
{
    int b[50];
    int i, j, k;
    i = first;
    j = mid + 1;
    k = first;
    while (i < mid && j <= last)
    {
        if (a[i] <= a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }
    while (i <= mid)
        b[k++] = a[i++];
    while (j <= last)
        b[k++] = a[j++];
    for (i = first; i <= last; i++)
        a[i] = b[i];
}

```

Output: enter the limit 5

enter the elements 10 45 35 5 55

sorted elements are 5 10 35 45 55

Lab Program -18

(18) C program to find minimum and maximum among given list

Algorithm:

Step 1: Start

Step 2: Input $n, i, a[10], \max = 0, \min = a[0]$

Step 3: for $i = 0$ to n

Step 4: for $i = 0$ to n

if $(a[i] > \max)$

$\max = a[i]$

if $(a[i] < \min)$

$\min = a[i]$

end if

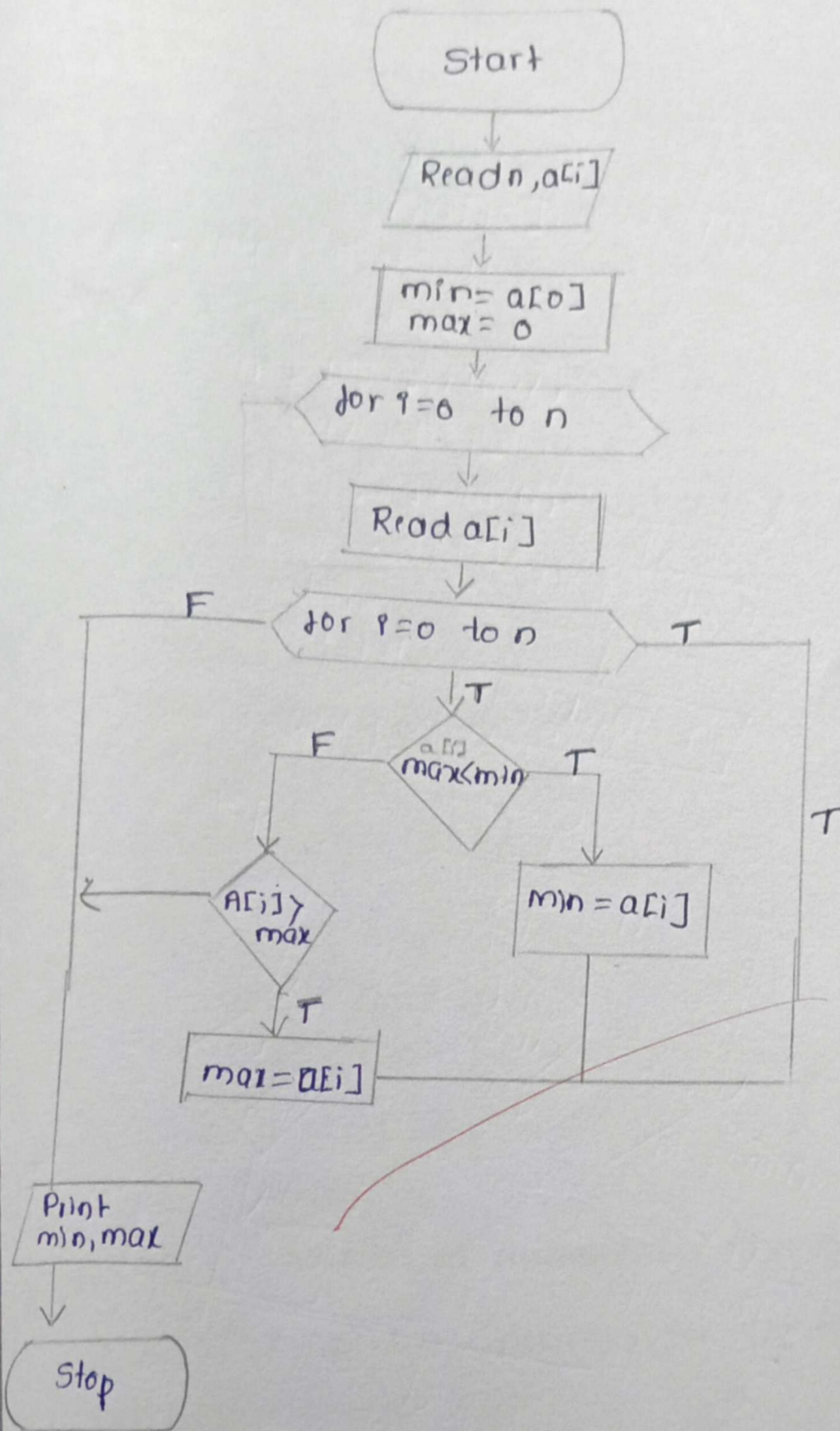
end for

Step 5: Display $\max, \min$

Step 6: Stop.

Lab Program -18

Flowchart



Lab Program - 18

⑮ Write a C program to find minimum and maximum among given list(set)

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int a[10], n, i, max, min;
```

```
printf("Enter the limit:");
```

```
scanf("%d", &n);
```

```
printf("Enter %d numbers:", n);
```

```
for (i = 0; i < n; i++)
```

```
scanf("%d", &a[i]);
```

```
max = 0;
```

```
min = a[0];
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
if (a[i] > max)
```

```
max = a[i];
```

```
if (a[i] < min)
```

```
min = a[i];
```

```
}
```

```
printf("Minimum number in the list is %d\n", min);
```

```
printf("Maximum number in the list is %d", max);
```

```
getch();
```

```
}
```


Lab Program - 18.

Output

Enter the limit : 5

Enter 5 (elements) numbers : 10

20

50

80

Minimum number in the list is 10

Maximum number in the list is 80

Ques
17-10-24

Lab Program - 19

(19) Write a C program to perform Quick sort

Algorithm

Step 1: Start

Step 2: Choosing a Pivot element

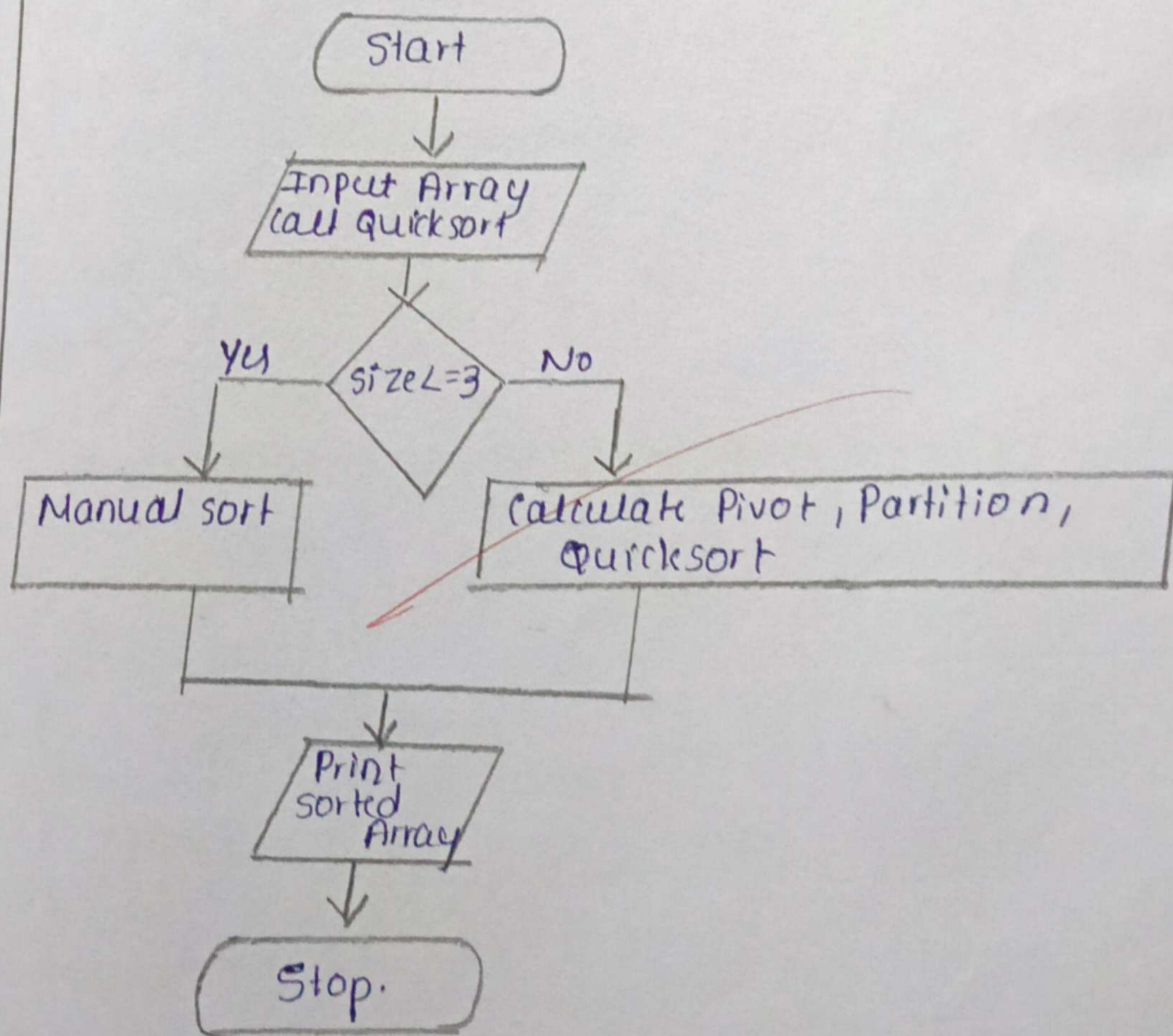
Step 3: Partitioning: We reorder the array so that all elements less than the pivot come before it and all elements greater than the pivot go after it

Step 4: Sorting the subarrays

Step 5: Final sorted Array

Step 6: Stop.

Flowchart



Lab Program - 19

① Write a C program to perform Quick sort

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i, n, a[25];
    printf("Enter the limit:");
    scanf("%d", &n);
    printf("Enter %d elements:", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    quicksort(a, 0, n-1);
    printf("The sorted order is:");
    for (i = 0; i < n; i++)
        printf("%d", a[i]);
    getch();
}

void quicksort(int a[25], int first, int last)
{
    int i, j, pivot, temp;
    if (first < last)
    {
        pivot = first;
        i = first;
        j = last;
        while (i < j)
        {
            while (a[i] <= a[pivot] && a[i] < last)
                i++;
            while (a[j] > a[pivot])
                j--;
            if (i < j)
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
        a[pivot] = a[j];
        a[j] = a[pivot];
        quicksort(a, first, j-1);
        quicksort(a, j+1, last);
    }
}
```



```

while (a[j] > a[pivot])
{
    j--;
    if (j < i)
    {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}
temp = a[pivot];
a[pivot] = a[j];
a[j] = temp;
quicksort(a, first, j-1);
quicksort(a, j+1, last);
}
}

```

Output

Enter the limit 5

Enter 5 elements 55

88

77

66

2

The sorted order is 2 55 66 77 88

Lab Program 20

Q. Write a C program for array counting using histogram

Algorithm:

Step 1: Start

Step 2: Input $n, arr[]$

Step 3: for ($i=1; i \leq n; i++$)

Step 4: if ($arr[i] > max\ x\ ele$)

$max\ ele = arr[i]$

Step 5: for ($i = max\ ele; i >= 0; i--$)

Print i

Step 6: for ($j=0; j \leq n; j++$)

Step 7: if ($arr[j] \geq i$)

Print '1'

else

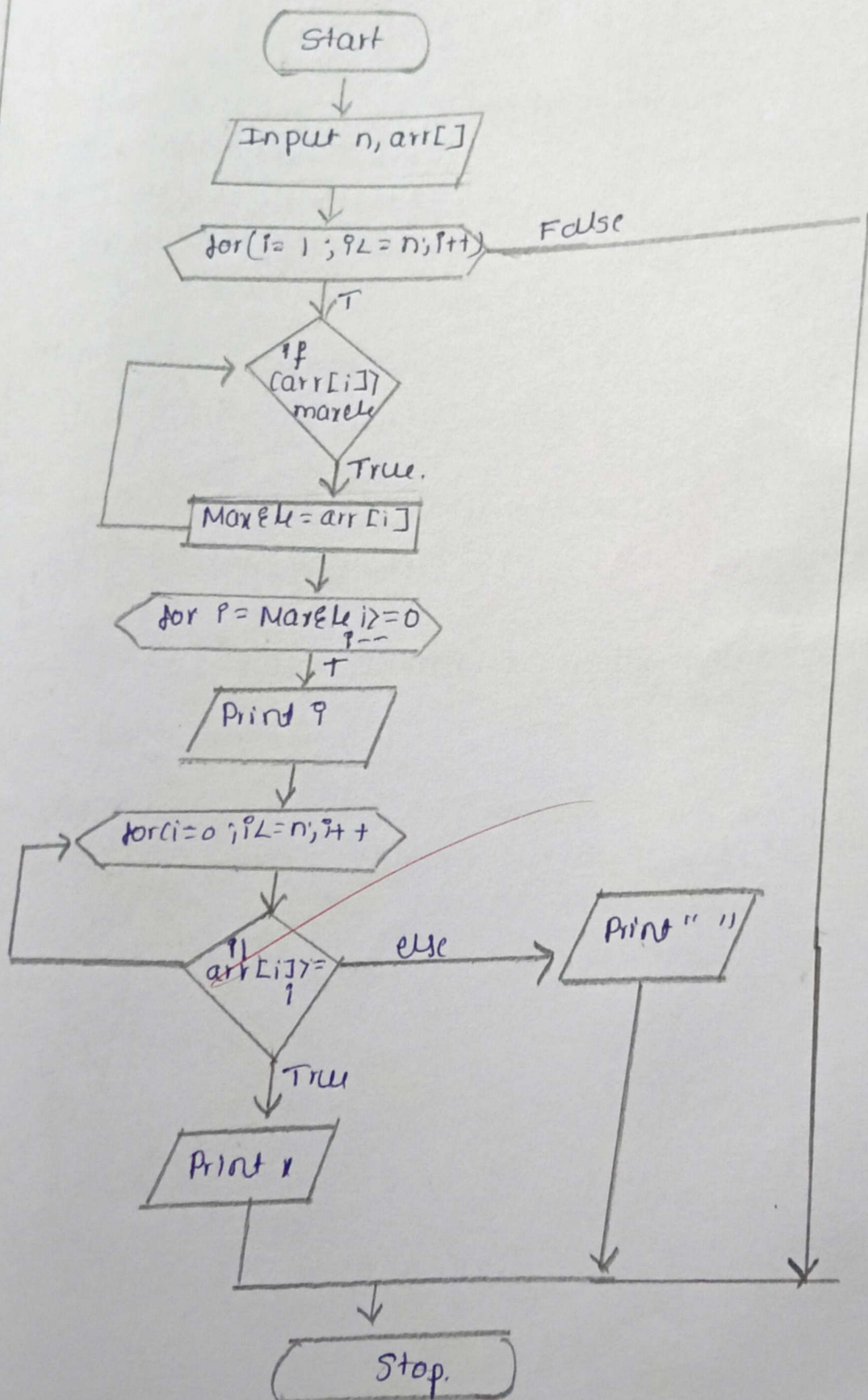
Print '0'

Step 8: Move to the next line

Step 9: Stop.

Lab Program - 20

Flowchart:



Lab Program - 20

```
#include <stdio.h>
```

```
void printHistogram (int arr[], int n) {
```

```
    int maxEle = arr[0];
```

```
    // find the maximum elements in the array
```

```
    for (int i = 1; i < n; i++) {
```

```
        if (arr[i] > maxEle) {
```

```
            maxEle = arr[i];
```

```
        }
```

```
    }
```

```
    // print histogram
```

```
    for (int i = maxEle; i > 0; i--)
```

```
    {
```

```
        printf("%2d |", i); // print the value.
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (arr[j] >= i)
```

```
            { printf("x"); // print 'x' in the current value  
              if (j >= i)
```

```
            }
```

```
        else
```

```
        { printf(" "); // print blank spaces if the  
          current value is < i
```

```
        }
```

```
    }
```

```
    printf("\n");
```

```
 }
```

```
}
```

```
int main() {
```

```
    int arr[] = {3, 5, 2, 6, 4};
```



```
int n = size of (arr) / size of (arr[0]);  
print Histogram (arr, n);  
return 0;
```

Output

61	x
51	x x
41	x x x
31	x x x x
21	x x x x x
11	x x x x x
01	x x x x x x

17-19-24